



Security Assessment & Incident Response Debrief

SecureCorp Customer Portal Engagement

BAGATHEESHWAR A · Junior Cybersecurity Consultant · August 2026

SecureCorp's Environment: High-Risk Security Posture

HIGH

Overall Risk Posture

Immediate remediation required

3

Major Web Vulnerability Classes

SQLi, XSS, Command Injection

1

Primary Automated Detection Path

SSH brute force only

4

Attack Simulations Performed

Full red-team exercise

Exposure

Weak authentication and unsanitized inputs enabled credential compromise and command execution.

Visibility

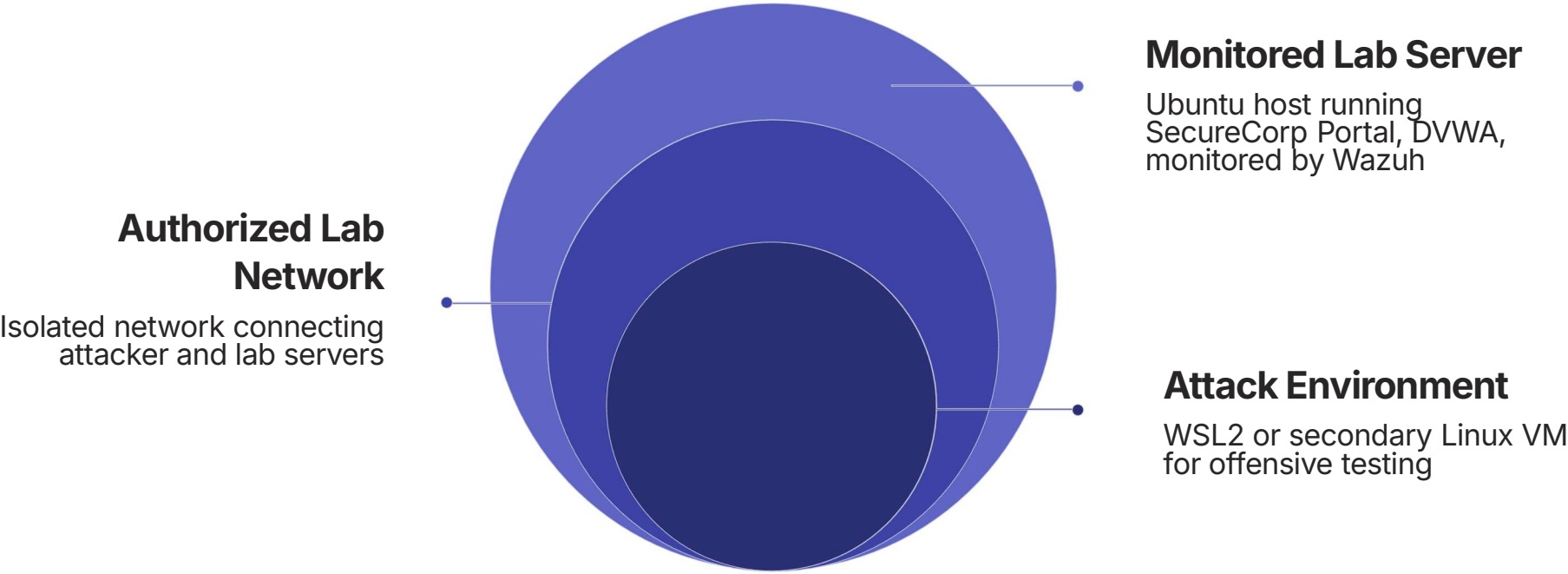
Wazuh collected authentication and Apache telemetry, but web-layer attacks required manual investigation.

Outcome

Risks addressable via MFA, secure coding, SIEM tuning, WAF/Fail2ban, least privilege, and patching.

Engagement Scope & Methodology

⚠ **Authorized Lab Only** — Testing restricted to self-owned isolated VMs and authorized test accounts. No production systems affected.



Authorized test environment: attacker machine → isolated lab network → Ubuntu target server → Wazuh SIEM monitoring layer.



Three Critical Application-Layer Attack Paths

SQL Injection — **CRITICAL**

Impact: Database contents and user password hashes exposed.

Root Cause: Unsanitized input passed into backend SQL queries.

Remediation: Parameterized queries (PDO) · Strict server-side validation

Cross-Site Scripting — **MEDIUM**

Impact: Reflected and Stored XSS executed — DOM manipulation and persistent script execution demonstrated.

Root Cause: Missing context-aware output encoding.

Remediation: Output encoding · HttpOnly cookies · Content Security Policy

Command Injection — **CRITICAL**

Impact: User-controlled input reached the OS ping utility — arbitrary command execution risk.

Root Cause: Unfiltered shell input.

Remediation: Strict allowlisting · Eliminate unsafe shell execution · Least privilege

❏ **Common Root Cause:** User input was trusted without sufficient server-side validation, parameterization, and output encoding.

Authentication Controls Were Easily Bypassed

SSH Service

Target: Ubuntu SSH

Method: Automated dictionary attack

Result: Valid test credentials recovered

Time: ~2 seconds

Web Application

Target: DVWA Web Login

Method: Automated HTTP brute-force

Result: Valid admin credentials recovered

Time: ~2 seconds

< 2 Seconds

to recover valid credentials on both attack surfaces — with no rate limiting, no account lockout, and no multi-factor authentication in place.

Weak Passwords

Dictionary-based credentials on SSH and web

No Rate Limiting

Unlimited login attempts permitted

No Account Lockout

No throttling after failed attempts

Single-Factor Auth

No MFA on admin interfaces



Priority Action: MFA + Strong Password Policy + Account Lockout + SSH Key Authentication

Wazuh: Strong Telemetry, Application-Layer Blind Spot

✓ Detected Automatically

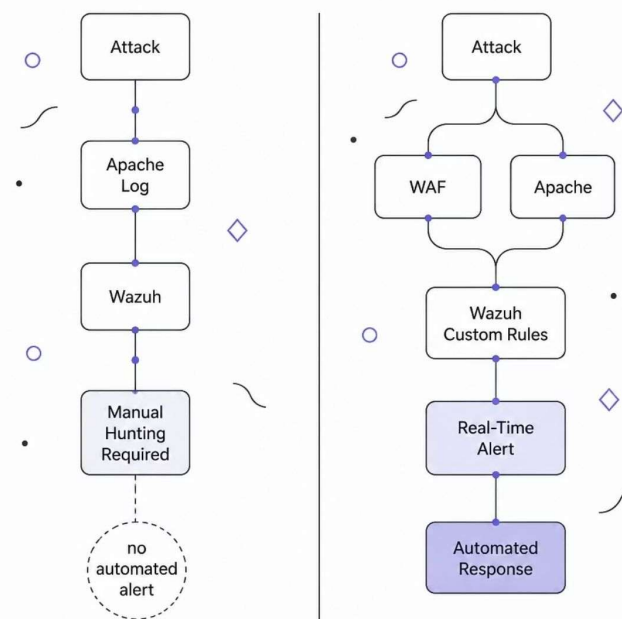
SSH Brute Force

Wazuh Rule 5720 — automatic high-severity alert generated without manual intervention.

⚠ Missed / Manual Hunt Required

- SQL Injection — no automated rule
- XSS (Reflected & Stored) — no automated rule
- Command Injection — no automated rule

Apache access logs contained evidence of all web attacks, but equivalent Wazuh decoders were not configured.



Key Message: Visibility ≠ Detection. Logs can contain full evidence of an attack without generating a single actionable alert.

Prioritized Remediation Roadmap

Recommended implementation sequence — not a committed schedule.

01 — Immediate

Authentication

- 14+ character passwords
- MFA for web admin
- Disable SSH password auth
- RSA/Ed25519 keys
- Account lockout

03 — 60 Days

Detection & Defense

- Custom Wazuh decoders/rules
- SQLi/XSS pattern detection
- Deploy WAF / ModSecurity
- Fail2ban / Active Response

02 — 30 Days

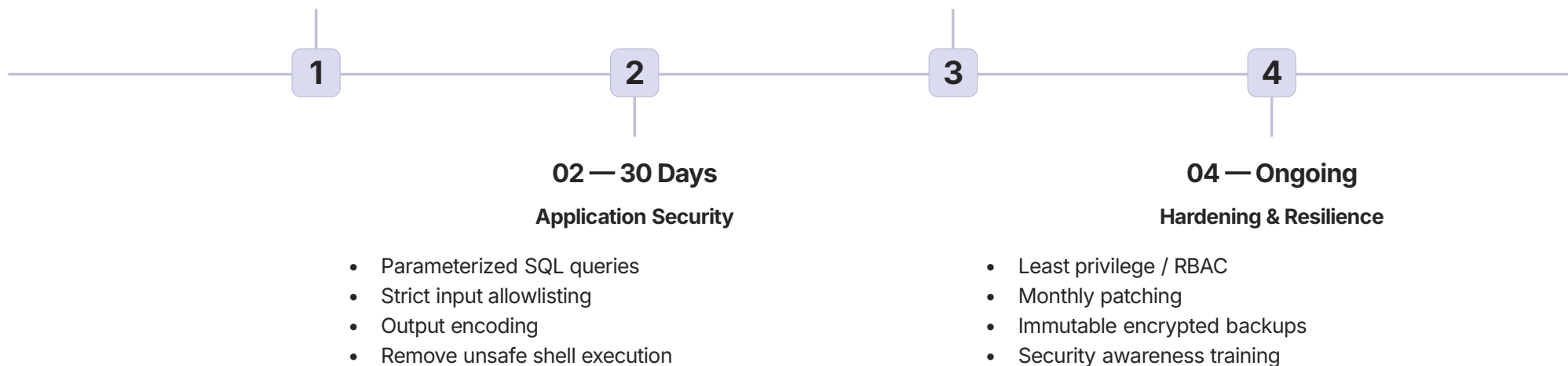
Application Security

- Parameterized SQL queries
- Strict input allowlisting
- Output encoding
- Remove unsafe shell execution

04 — Ongoing

Hardening & Resilience

- Least privilege / RBAC
- Monthly patching
- Immutable encrypted backups
- Security awareness training



Three Lessons From the Engagement

1

Secure by Default Is Not Enough

Applications require deliberate hardening and continuous assessment. Default configurations are starting points, not security postures.

2

Visibility $\neq$ Detection

Logs can contain full evidence of an active attack without generating a single actionable alert. Telemetry must be paired with tuned detection rules.

3

Security Must Be Layered

Authentication, secure coding, WAF, SIEM, least privilege, patching, backups, and user awareness reinforce each other. No single control is sufficient.

 **Key Takeaway:** Eliminate the critical attack paths first, then close the monitoring gap. Both are required for measurable risk reduction.

CLOSE

Questions & Discussion

From Findings → To Fixes → To Measurable Risk Reduction

1

What remediation should be implemented first?

2

How can patching be performed with minimal disruption?

3

How do we prevent recurring secure-coding mistakes?

4

How will we validate that Wazuh detects web attacks after tuning?